

• 8 PRODUCTS • OPEN SOURCE • ONE STACK

Infrastructure that doesn't stop when the **connection** does.

Eight Zyvor products, each built for a site that has to keep running on its own — from the hardware in the rack to the AI brain in the cluster. This is how they fit together.

8

products in the stack

8

now all under the zyvorai org

0

cloud dependency to keep a site running

A→N→A→A→V

the durable loop every event ends in

THE STACK, IN ORDER

01

Device Agent

02

Nodra

03

Fleet

04

OTA

05

relay-edge

06

relay-pubsub

07

Zyvor Relay

08

Zynera

01



Zyvor Device Agent

Discovers the box, first

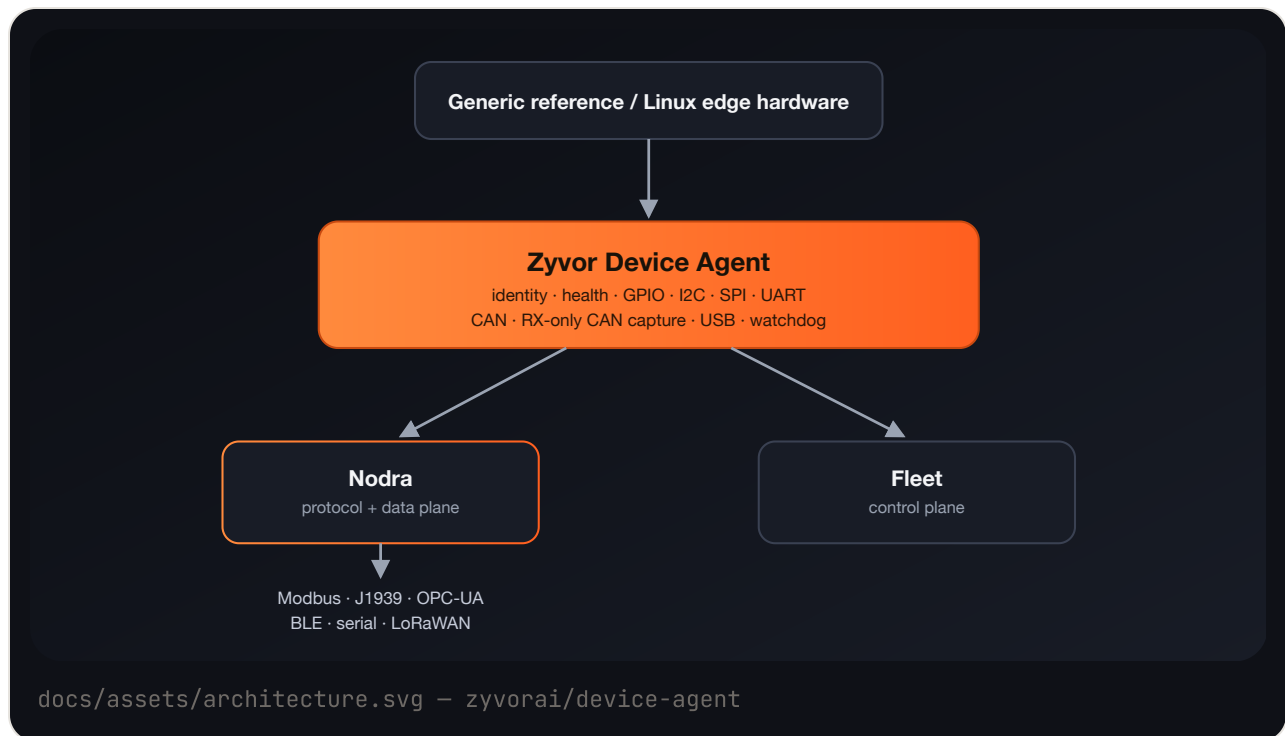
v0.1.6

zyvorai/device-agent

Rust · arm64/amd64

“Linux hardware edge agent for Zyvor — discover the box, expose physical interfaces, publish to Nodra, expose Fleet-compatible inventory.”

- Discovers identity, health, GPIO, I2C, SPI, UART, CAN, USB, and watchdog state on real Linux hardware
- Deliberately stays out of protocol semantics — decoding `Modbus` / `OPC-UA` / `BLE` is Nodra's job, not this agent's
- arm64-first: built for the actual boxes running at the edge, not a server rack
- Marked “**production-hardening**” — past prototype, not yet declaring 1.0



02

**Nodra**

Runs the site, with or without the cloud

v0.2.2

zyvorai/nodra

public

“The open edge runtime that keeps sites running when the cloud doesn't.”

- Local `MQTT` / `HTTP` ingress in front of a durable, WAL-backed store-and-forward buffer
- Device twins and edge-app reconciliation, offline-first by design — not offline-tolerant

- Publishes a direct, named comparison against Node-RED, EMQX/HiveMQ Edge, AWS Greengrass, and Azure IoT Edge in its own docs
- Honest about limits: v0.2 is single-writer by design, HA is roadmap not shipped — CI runs multi-hour soak tests on a real lab host

03



Zyvor Fleet

Holds the whole fleet together over a bad WAN

v0.3.0

zyvorai/fleet

Go 1.27+

“Every site. Still running.” — offline-first edge fleet control plane for Linux, Kubernetes, containers, and virtual machines.

- The `fleet-agent` caches its full desired-state revision locally and keeps reconciling straight through a WAN outage
- Targets systemd, containers, k3s, QEMU/KVM, and GPU edge servers from one control plane
- No arbitrary remote shell exposed — a deliberately narrower attack surface than a generic RMM tool
- Its own README carries the authoritative map of how this whole product family fits together

04



Zyvor OTA

Updates the OS underneath everything else

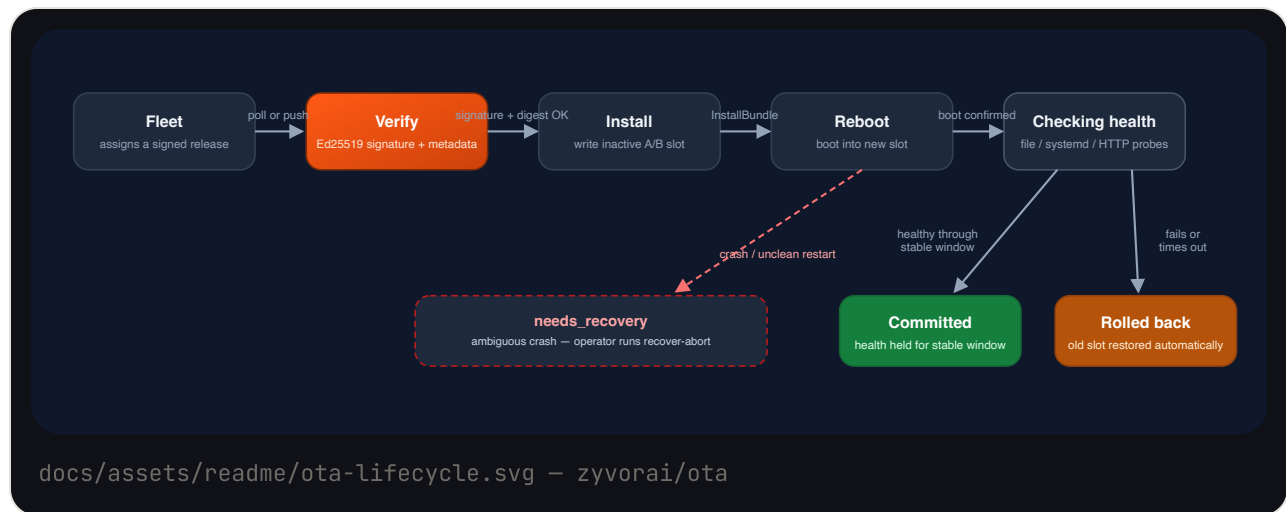
v0.1.0

zyvorai/ota

Go 1.27.1

“Signed, recoverable device OS updates for the Zyvor Platform.”

- RAUC-managed A/B slot installs with **Ed25519-signed** bundles
- Automatic health-check-gated rollback — a bad update reverts itself, no one has to notice first
- An explicit `needs_recovery` interlock for the one ambiguous case: a crash mid-update that isn't safe to auto-resolve
- Names its own comparison set in its docs: Mender, SWUpdate, balena



05



relay-edge

The upstream brain for Zyvor Relay

v0.1.2

zyvorai/relay-edge

Go 1.27+ · Apache-2.0

A synthetic IoT/farm site & event simulator — stamps realistic site topology onto four built-in simulators and publishes it into Zyvor Relay, so the rest of the stack has real traffic before real hardware shows up.

- Four simulators: a **47-point** NFPA-style fire-water plant, a **77-device / 18-class** fleet catalog, a **24-asset** remote-edge NOC, and a farm domain
- Every event is stamped with season/site/zone/recipients/verification probe before Relay ever sees it
- Three live browser control rooms, streamed over Server-Sent Events
- Publishes direct to Relay or through `relay-pubsub` — the same two paths every other producer in this stack uses

Lab Configure Logs Fire-water Remote edge Fleet Docs

relay-edge

Stamp the site. Publish the event.

The upstream companion for Zyvor Relay — store domain context, send stamped events, receive live outcomes.

[Test lab](#)[Configure](#)[Setup guide](#)

FIRST INSTALL

Get started

Wire this edge to a reachable pubsub or Relay host, probe, then prove the lab. Peers may be remote — do not use laptop 127. 0. 0. 1 unless everything runs here.

- Health** done
Confirm this server answers /healthz.
- Auth** done
If auth is required, paste edge_api_token in Configure (browser only).
- Wire publish** done
Set gateway_base_url (pubsub) or clear it and set relay_base_url. Add JWT; enable TLS insecure for lab certs. Save.
- Probe** —
Run Probe backends — edge plus gateway and/or Relay must be reachable.

/ui — live capture, this session

06



relay-pubsub

Speaks Pub/Sub so nothing else has to

[v0.4.0](#)[zyvorai/relay-pubsub](#)[public GHCR](#)

“Google Cloud Pub/Sub compatibility for Zyvor Relay.”

- Implements the real gRPC/REST surface: topics, subscriptions, Publish, Pull, StreamingPull, Ack, Seek, Snapshots, a subset of IAM, SchemaService
- Ships a second, in-memory backend built just for demos and CI
- Its production backend forwards straight into Relay's own `/v1/events`
- Public, versioned images at `ghcr.io/zyvorai/relay-pubsub`

07



Zyvor Relay

The loop everything else in this stack feeds

[private](#)[zyvorai/relay](#)[continuously deployed](#)

“On-prem and edge reliability control plane for durable events, acknowledged notifications, idempotent actions, and verified outcomes.”

"Never lose an accepted event. Never knowingly execute a critical side effect twice. Never call an action successful until the expected outcome is verified."

- The loop: **Accept → Notify → Ack → Act → Verify**
- Multi-tenant, with fully isolated data and dispatch per tenant
- Transactional outbox/inbox, leased jobs with a dead-letter queue you can list and replay, circuit breakers with exponential backoff
- Custom JWT auth with per-token revocation — logout is immediate, not TTL-delayed
- An **L0–L13** verification suite, plus a Prometheus **/metrics** endpoint
- Its own operator console, compiled straight into the Go binary

08



Zynera

The optional AI/GPU brain, when a site needs one

v1.0.0

zyvorai/zynera

private

formerly "Forge"

"The Kubernetes brain for AI infrastructure" — an enterprise control plane for GPU clusters.

*"VMware vCenter for AI/GPU infrastructure + OpenShift-grade Kubernetes ops
+ an AI SRE that never sleeps."*

- Seven operators and **75+ CRDs** running the real GPU scheduling, via eBPF
- An embedded AI operations engineer — **Zeus** — watching the cluster
- Twelve capability domains by its own count: placement, FinOps, security/SIEM, network intelligence, a model catalog, RAG studio, federation/DR, live vGPU migration, and more
- Steps in only when a site runs real AI/GPU workloads — every other product in this stack works without it

How a single event actually moves through this

Not a diagram of intentions — this is the real path, drawn from what each product's own docs say it hands off to next.

- 01 **Device Agent** discovers the hardware and hands off — protocol and data-plane work to **Nodra**, lifecycle and control-plane work to **Fleet**.
- 02 **Nodra** (and, for synthetic/dev traffic, **relay-edge**) stamp and publish events — direct, or through **relay-pubsub**'s Pub/Sub-compatible gateway.
- 03 **Zyvor Relay** runs the durable loop — `Accept → Notify → Ack → Act → Verify` — on every event, regardless of which producer sent it.
- 04 When a site runs AI/GPU workloads, Relay can gate a critical act behind a **Zynera** Decision Record — human freeze/attest, in Zeus, before Act proceeds.
- 05 Underneath all of it, **Fleet** assigns the signed releases that **OTA** installs, verifies, and — if health checks fail — rolls back on its own.



8 products · 6 public, 2 private · all zyvorai

Every quote, version tag, and stat on this page is pulled directly from each product's own README and docs as of September 2026 — nothing here is invented. Diagrams are the products' own, inlined as-is.